

Category B Problems

5 problems · largest volume in a typical session · full 5-step walkthrough, complexity depth varies by item · OOP, Classes and Objects — Week 3

M1. From Parallel Arrays to a Class

Scenario

The T&P cell currently tracks student placements using three parallel arrays — names, companies, packages — and it has already gone out of sync twice this month. Rebuild it the OOP way.

Task

- Define a class PlacementRecord with fields String studentName, String company, and double packageLpa. • Write a constructor that sets all three fields.
- Write an instance method printRecord() that prints one formatted line.
- Create three PlacementRecord objects for different students, store them in an array of PlacementRecord, and print each one in a loop.

Suggested method signature(s)

```
class PlacementRecord { ... }  
void printRecord()
```

Sample Input / Output

Ravi, TCS, 4.5 Anitha, Zoho, 6.2 Karthik, Infosys, 4.0	Ravi -> TCS @ 4.5 LPA Anitha -> Zoho @ 6.2 LPA Karthik -> Infosys @ 4.0 LPA

Concepts covered: OOP vs parallel arrays, constructors, arrays of objects, instance methods.

M2. Encapsulated Mess-Card Wallet

Scenario

A hostel mess-card top-up wallet must never be allowed to go negative, and its balance must never be overwritten directly from outside the class.

Task

- Define a class MessWallet with a private double balance field.
- Write a public constructor that accepts an opening balance; if a negative value is given, start at 0 instead and print a warning.
- Write a public method topUp(double amount) that rejects amount <= 0 with a message, otherwise adds it to the balance.
- Write a public method deduct(double amount) that rejects any amount greater than the current balance,

printing a clear message instead of allowing balance to go negative.

- Write a public `getBalance()` method for read-only access. There must be no public way to set balance directly from outside the class.

Suggested method signature(s)

```
void topUp(double amount)
void deduct(double amount)
double getBalance()
```

Sample Input / Output

<i>opening = 500 topUp(200) deduct(1000)</i>	<i>Balance after top-up: 700.0 Deduct rejected: insufficient balance Final balance: 700.0</i>

Concepts covered: Encapsulation, private fields, validation inside methods, controlled read-only access.

M3. Overloaded Constructors for a Course

Scenario

Some courses come with a separate lab component and lab credit count; most don't. Support both without writing the same setup logic twice.

Task

- Define a class `Course` with fields `code`, `title`, `credits`, and `labCredits`.
- Write a constructor `Course(String code, String title, int credits, int labCredits)` that sets all four fields directly.
- Write a second constructor `Course(String code, String title, int credits)` for theory-only courses, which uses `this(...)` to chain to the four-argument constructor above with `labCredits` set to 0.
- Write a method `totalCredits()` returning `credits + labCredits`.
- Create one `Course` object with each constructor and print `totalCredits()` for both.

Suggested method signature(s)

```
public Course(String code, String title, int credits, int labCredits)
public Course(String code, String title, int credits) // chains via this(...)
```

Sample Input / Output

<i>21CSC201J, Data Structures, 4 (theory-only) 21CSC205L, DSA Lab, 3, LabCredits 1</i>	<i>21CSC201J total credits: 4 21CSC205L total credits: 4</i>

Concepts covered: Constructor overloading, this() chaining, avoiding duplicated initialization logic.

M4. Reference Copies and a Shared ID Card

STEP SEM-3 · Week 3 · OOP Practice Problems · Category B Page 2 of 4

CodInClub
Powered by BridgeLabz

Scenario

Two “different” variables in a script both claim to represent Ravi's library card, but only one of them is actually real. Prove it with code.

Task

- Define a class IdCard with fields String name and int booksIssued, both set through a constructor. • Create one IdCard object for Ravi. Assign a second variable to point at that same object (do not create a new one).
- Through the second variable, change booksIssued.
- Print the field's value as seen through the first variable, and print whether the two variables are == to each other.
- Create a third, separate IdCard object with identical field values, and print whether it is == to the first.

Suggested method signature(s)

```
class IdCard { String name; int booksIssued; }
```

Sample Input / Output

<pre>IdCard ravi = new IdCard("Ravi", 0); IdCard duplicate = ravi; duplicate.booksIssued = 3; IdCard separate = new IdCard("Ravi", 3);</pre>	<pre>Ravi's booksIssued (via first variable): 3 duplicate == ravi: true separate == ravi: false</pre>
--	---

Concepts covered: Reference copying, ==, object identity vs content equality.

M5. Instance vs Static: Splitting a Class Correctly

Scenario

A club member's first draft of Student stores attendance, name, AND the college name as instance fields — meaning every single student object ends up with its own copy of “SRM Institute of Science and Technology” typed in separately. Fix the design.

Task

- Define a class Student with instance fields name and attendance, and a static field collegeName shared by every student.
- Add a static field studentCount that increments once inside the constructor, every time. • Write a static method

printCollegeInfo() that prints collegeName and studentCount — and must not reference any instance field.

- Create two Student objects, then call printCollegeInfo() through the class name, not through either object.

Suggested method signature(s)

```
static void printCollegeInfo()
```

Sample Input / Output

<i>2 Student objects created</i>	<i>SRM Institute of Science and Technology Students created: 2</i>

Concepts covered: Instance vs static members, why static methods cannot touch instance fields, accessing static members via the class.

